

UNIVERSIDAD REGIONAL AUTÓNOMA DE LOS ANDES (UNIANDES)

Departamento de Desarrollo de Software | Facultad de Sistemas

Programación de Backend y MCP en Python para IA Generativa

Estudiante: Juan Carlos Lozada | Fecha: 31 de agosto de 2026 (Lunes)

Informe de Entrega — Practica 2 / Sesion 3

Curso: Programacion de Backend y MCP en Python para IA Generativa

Tema: Vibe Coding: El Experimento Guiado

Estudiante: Juan Carlos Lozada

Institucion: Universidad Regional Autonoma de Los Andes (UNIANDES)

Departamento: Desarrollo de Software

Fecha de ejecucion: 31 de agosto de 2026 (Lunes)

1. Introduccion y Objetivo del Experimento

El proposito de esta practica fue someter a prueba la metodologia de *vibe coding* (generacion de codigo mediante peticiones directas en lenguaje natural sin especificacion previa). A traves de tres rondas progresivas y un bloque de cierre, se contrasto la prediccion individual con la salida generada por el modelo de IA en la maquina local y con la solucion proyectada por el instructor.

El experimento busca evidenciar de forma practica como la ambigüedad de los requerimientos produce discrepancias funcionales, decisiones arbitrarias por parte del modelo y fallos de regresion en tiempo de ejecucion.

2. Registro de Rondas y Evidencias

Ronda 1 — Validador de contraseñas

- Peticion realizada:** hazme un validador de contraseñas
- Casos de prueba evaluados:** '12345', 'password', '' (cadena vacia).

```
=== RONDA 1: Validador de contraseñas ===
Entrada: '12345'    -> Valido: False | Mensaje: La contraseña debe tener al menos 6 car
acteres.
Entrada: 'password' -> Valido: False | Mensaje: La contraseña debe contener al menos un
digito numerico.
Entrada: ''        -> Valido: False | Mensaje: La contraseña no puede estar vacia.
```

Tabla Comparativa de la Ronda 1:

| Variable de control | Prediccion personal | Resultado obtenido en local | Resultado del instructor |

```

|---|---|---|---|
| Longitud minima | 8 caracteres | 6 caracteres | 8 caracteres |
| Complejidad requerida | Mayuscula + digito | Solo digito | Mayuscula + digito + simbolo |
| Manejo de cadena vacia | ValueError o booleano False | Tupla (False, msg) | Booleano False
|

```

Ronda 2 — Agregacion de validacion de email

- **Peticion realizada:** agregale que tambien valide un formato de email
- **Casos de prueba evaluados:** 'ana@correo.com', 'ana@', ''

```

=== RONDA 2: Validacion de email ===
Entrada: 'ana@correo.com' -> Valido: True | Mensaje: Email valido.
Entrada: 'ana@'          -> Valido: False | Mensaje: Formato de correo no valido: 'ana
@'
Entrada: ''              -> Valido: False | Mensaje: El email no puede estar vacio.

```

Tabla Comparativa de la Ronda 2:

```

| Cuestion evaluada | Prediccion | Resultado real |
|---|---|---|
| Mantenimiento del contrato de salida | Si, tupla (bool, str) | Se mantuvo la tupla, pero el
modelo agrego pruebas no solicitadas en consola |
| Tratamiento de formato incompleto ( ana@ ) | Rechazo por expresion regular | Rechazado con
mensaje especifico de formato no valido |

```

Ronda 3 — Coleccion de usuarios en memoria

- **Peticion realizada:** ahora que maneje una lista de varios usuarios, cada uno con su contraseña y su email
- **Casos de prueba:** Registro de dos cuentas institucionales (carlos@uniandes.edu.ec y maria@uniandes.edu.ec).

```

=== RONDA 3: Lista de varios usuarios ===
Registro 1: {'resultado': 'ok', 'usuario': {'email': 'carlos@uniandes.edu.ec', 'password': 'Clave123', 'rol': 'usuario'}}
Registro 2: {'resultado': 'ok', 'usuario': {'email': 'maria@uniandes.edu.ec', 'password': 'Pass456', 'rol': 'usuario'}}
Lista completa (2 usuarios): [{'email': 'carlos@uniandes.edu.ec', 'password': 'Clave123', 'rol': 'usuario'}, {'email': 'maria@uniandes.edu.ec', 'password': 'Pass456', 'rol': 'usuario'}]

```

Analisis de regresion:

La transicion hacia una lista provoco que la validacion ya no pueda invocarse de forma

desacoplada para un unico campo, obligando a interactuar a traves de una estructura de clase con respuestas basadas en diccionarios, sin incorporar control de llaves primarias o correos duplicados.

Bloque de Cierre — Inyeccion de fallo por instruccion ambigua

- **Peticion realizada:** ahora que la validacion de contrasena sea opcional para usuarios administradores

Resultado en ejecucion:

```
=== BLOQUE DE CIERRE: Falla provocada ===
Admin registrado sin clave: {'resultado': 'ok', 'usuario': {'email': 'admin@uniandes.edu.ec', 'password': None, 'rol': 'admin'}}
Intentando autenticar al administrador con contrasena 'admin123'...
ERROR CAPTURADO (Inconsistencia en runtime): Fallo de integridad: El usuario admin 'admin@uniandes.edu.ec' no tiene contrasena asignada.
```

Diagnostico tecnico del incidente:

Al relajar la regla sin especificar el comportamiento esperado en autenticacion ni la persistencia de credenciales, el modelo permitio asignar `password=None`. Al momento de contrastar credenciales en el metodo `autenticar()`, el codigo intenta procesar un valor nulo, detonando un fallo critico de integridad que dejaria inoperativo el inicio de sesion de los administradores o permitiria bypass si se evaluara con logica debil.

3. Respuestas a la Reflexion Final

1. **¿En que ronda tu prediccion se alejo mas de lo que realmente paso? ¿Por que crees que fallaste esa prediccion?**

La mayor desviacion ocurrio en la Ronda 1. Se predijo una longitud estandar de 8 caracteres con politicas de complejidad habituales (mayusculas y digitos), pero el modelo resolvió fijar un umbral de 6 caracteres requiriendo unicamente digitos. Esta falla se debe a que la peticion carecia de una especificacion de dominio (*Domain Specification*), obligando al motor generativo a inferir parametros bajo su propia probabilidad estadística.

2. **¿Tu resultado fue exactamente igual al del instructor en todas las rondas, o hubo diferencias con el mismo pedido? ¿Que te dice eso sobre pedir cosas sin spec?**

Existieron discrepancias directas con el resultado del instructor desde la primera instruccion: tanto en las reglas de validacion de contrasena como en la forma de retornar los errores y en el diseno de clases posterior. Esto demuestra que programar sin especificacion introduce variabilidad no controlada, impidiendo la reproducibilidad, las pruebas formales y la coherencia en un equipo de desarrollo.

3. **Sintesis de cierre:**

"Si yo tuviera que darle este codigo a otra persona manana, tendria que explicarle que criterios

exactos determinan una contraseña valida, por que no se verifican correos duplicados y como debe autenticarse un administrador sin contraseña, porque eso no esta escrito en ningun lado."

4. Conclusiones

- El desarrollo asistido por IA requiere pasar de prompts conversacionales ambiguos a especificaciones de software estructuradas (*Spec-Driven Development*).
- La omision de casos borde en la instruccion inicial genera deuda tecnica inmediata y errores en tiempo de ejecucion que dificultan el mantenimiento.